

✓ Лабораторна робота № 5

Тема: Автокорекція тексту з використанням мінімальної відстані редагування

Варіант 2

Студентка: Гріценко Аннастасія

Група I-22

```
import re
import numpy as np
import json
import collections
from collections import Counter
```

✓ ЗАВДАННЯ 1: Завантаження корпусу

```
def load_and_process_arxiv(file_path='arxiv-metadata-oai-snapshot.json', limit=30000):
    """
    Завантажує анотації наукових статей з датасету ArXiv.
    Оскільки файл дуже великий (JSON Lines), зчитуємо лише перші 'limit' записів.

    Якщо файл не знайдено, використовує вбудований науковий корпус NLTK (як резерв).
    """
    print(f"Завантаження корпусу наукових текстів (limit={limit} статей)...")
    words = []

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            for i, line in enumerate(f):
                if i >= limit:
                    break
                article = json.loads(line)
                # Використовуємо заголовок та анотацію (abstract)
                text_content = article.get('title', '') + " " + article.get('abstract', '')

                # Приводимо до нижнього регістру
                text_content = text_content.lower()

                # Видаляємо LaTeX формули (проста евристика: видаляємо все між $)
                text_content = re.sub(r'\$.?*\$', '', text_content)

                # Знаходимо слова (літери, цифри, без пунктуації) [cite: 5]
                tokens = re.findall(r'[a-z]+' , text_content)
                words.extend(tokens)

        print(f"Оброблено {limit} статей з ArXiv.")

    except FileNotFoundError:
        print(f"Файл '{file_path}' не знайдено. Використовую резервний корпус (NLTK Brown - Science).")
        import nltk
        nltk.download('brown', quiet=True)
        from nltk.corpus import brown
        # Категорії наукового стилю: news, editorial, reviews, learned
        words = [w.lower() for w in brown.words(categories=['learned', 'reviews'])]

    return words

def get_count(word_l):
    """Створює словник частотності слів [cite: 5, 56-66]."""
    return Counter(word_l)
```

✓ ЗАВДАННЯ 2 & 3: Ймовірності та Операції редагування

```
def get_probs(word_count_dict):
    """Обчислює ймовірності слів P(c) [cite: 7, 67-75]."""
    probs = {}
    total_count = sum(word_count_dict.values())
    for word, count in word_count_dict.items():
        probs[word] = count / total_count
    return probs

# Базові операції редагування [cite: 6, 25-33, 77-118]
```

```
def delete_letter(word):
    split_l = [(word[:i], word[i:]) for i in range(len(word))]
    return [L + R[1:] for L, R in split_l if R]

def switch_letter(word):
    split_l = [(word[:i], word[i:]) for i in range(len(word))]
    return [L + R[1] + R[0] + R[2:] for L, R in split_l if len(R) > 1]

def replace_letter(word):
    letters = 'abcdefghijklmnopqrstuvwxyz'
    split_l = [(word[:i], word[i:]) for i in range(len(word))]
    return [L + c + R[1:] for L, R in split_l if R for c in letters if c != R[0]]

def insert_letter(word):
    letters = 'abcdefghijklmnopqrstuvwxyz'
    split_l = [(word[:i], word[i:]) for i in range(len(word) + 1)]
    return [L + c + R for L, R in split_l for c in letters]
```

ЗАВДАННЯ 4: Кандидати на відстані 1 та 2

```
def edit_one_letter(word, allow_switches=True):
    """Слова на відстані 1 редагування [cite: 8, 120-130]."""
    edit_set = set()
    edit_set.update(delete_letter(word))
    edit_set.update(replace_letter(word))
    edit_set.update(insert_letter(word))
    if allow_switches:
        edit_set.update(switch_letter(word))
    return edit_set

def edit_two_letters(word, allow_switches=True):
    """Слова на відстані 2 редагувань [cite: 8, 131-138]."""
    edit_set = set()
    one_edit = edit_one_letter(word, allow_switches)
    for w in one_edit:
        edit_set.update(edit_one_letter(w, allow_switches))
    return edit_set
```

ЗАВДАННЯ 5: Min Edit Distance (DP)

```
def min_edit_distance(source, target, ins_cost=1, del_cost=1, rep_cost=2):
    """
    Обчислює мінімальну відстань редагування (алгоритм Вагнера-Фішера) [cite: 9, 34-41, 174-197].
    """
    m = len(source)
    n = len(target)
    D = np.zeros((m+1, n+1), dtype=int)

    for row in range(m+1):
        D[row, 0] = row * del_cost
    for col in range(n+1):
        D[0, col] = col * ins_cost

    for row in range(1, m+1):
        for col in range(1, n+1):
            r_cost = rep_cost
            if source[row-1] == target[col-1]:
                r_cost = 0

            D[row, col] = min(
                D[row-1, col] + del_cost,
                D[row, col-1] + ins_cost,
                D[row-1, col-1] + r_cost
            )
    return D, D[m, n]
```

ЗАВДАННЯ 6: Система автокорекції

```
def get_corrections(word, probs, vocab, n=2, verbose=False):
    """Повертає топ-n корекцій для слова [cite: 10, 140-172]."""
    suggestions = []

    # 1. Чи є слово правильним?
    if word in vocab:
        suggestions = [word]
    # 2. Перевірка відстані 1
    if not suggestions:
```

```

        suggestions = list(edit_one_letter(word) & vocab)
# 3. Перевірка відстані 2
if not suggestions:
    suggestions = list(edit_two_letters(word) & vocab)
# 4. Якщо нічого не знайдено - повертаємо вхідне слово
if not suggestions:
    suggestions = [word]

# Сортвання за ймовірністю P(c)
best_words = {}
for w in suggestions:
    best_words[w] = probs.get(w, 0)

n_best = sorted(best_words.items(), key=lambda x: x[1], reverse=True)[:n]

if verbose:
    print(f"Input: {word} -> Suggestions: {n_best}")

return n_best

```

▼ ЗАВДАННЯ 7: Оцінка та Головний блок

```

def evaluate_model(test_data, probs, vocab):
    """Оцінює точність системи[cite: 11]."""
    correct_predictions = 0
    total = len(test_data)

    print(f"\n--- Оцінка точності ({total} слів) ---")
    for misspelled, correct in test_data:
        pred = get_corrections(misspelled, probs, vocab, n=1)[0][0]
        is_correct = (pred == correct)
        if is_correct:
            correct_predictions += 1
        else:
            print(f"MISS: '{misspelled}' -> '{pred}' (Expected: '{correct}')")

    return correct_predictions / total

```

▼ ГОЛОВНИЙ БЛОК ВИКОНАННЯ

```

if __name__ == "__main__":
    # 1. Завантаження даних
    # Примітка: переконайтесь, що файл archiv-metadata-oai-snapshot.json доступний,
    # або код переключиться на NLTK Brown corpus.
    word_l = load_and_process_archiv()

    if word_l:
        vocab = set(word_l)
        word_count_dict = get_count(word_l)
        probs = get_probs(word_count_dict)

        print(f"Розмір словника: {len(vocab)} унікальних слів.")
        print(f"Топ слів: {word_count_dict.most_common(5)}")

        # 2. Демонстрація корекції (Наукова лексика)
        print("\n--- Демонстрація автокорекції (Scientific Domain) ---")
        my_word = "algorithm" # algorithm
        res = get_corrections(my_word, probs, vocab, verbose=True)

        my_word = "network" # network
        res = get_corrections(my_word, probs, vocab, verbose=True)

        my_word = "theorem" # theorem
        res = get_corrections(my_word, probs, vocab, verbose=True)

        # 3. Демонстрація Min Edit Distance
        print("\n--- Min Edit Distance ---")
        s, t = "neural", "natural"
        _, dist = min_edit_distance(s, t)
        print(f"Dist('{s}', '{t}') = {dist}")

        # 4. Оцінка точності
        # Тестовий набір специфічний для наукових статей
        test_set = [
            ('algorithm', 'algorithm'),
            ('analysis', 'analysis'),
            ('dataa', 'data'),
            ('hypothesis', 'hypothesis'),
            ('experment', 'experiment'),

```

```
    ('distribtion', 'distribution'),
    ('proobability', 'probability'),
    ('variabl', 'variable'),
    ('funcction', 'function'),
    ('learnng', 'learning')
]

acc = evaluate_model(test_set, probs, vocab)
print(f"-----")
print(f"Точність моделі (Scientific): {acc * 100:.2f}%")
print(f"-----")
```

Завантаження корпусу наукових текстів (limit=30000 статей)...

Файл 'arxiv-metadata-oai-snapshot.json' не знайдено. Використовую резервний корпус (NLTK Brown - Science).

Розмір словника: 19284 унікальних слів.

Топ слів: [('the', 14907), ('', 10560), ('of', 8792), ('.', 8322), ('and', 5443)]

--- Демонстрація автокорекції (Scientific Domain) ---

Input: algoritm -> Suggestions: [('algorithm', 4.49252443933295e-06)]

Input: networkk -> Suggestions: [('network', 1.79700977573318e-05)]

Input: theorum -> Suggestions: [('theorem', 8.08654399079931e-05)]

--- Min Edit Distance ---

Dist('neural', 'natural') = 3

--- Оцінка точності (10 слів) ---

Точність моделі (Scientific): 100.00%
